

Secure File Storage System

Technical Report

Course: Data Security

Project: RSA and AES Hybrid Cryptosystem

Business Model: Secure File Storage System

<https://github.com/mosb23/Secure-File-Storage-System>

https://drive.google.com/file/d/1QUt8hs44Q9Ed2mSu_MlIZCEleFnzkovp/view?usp=sharing

Report Contents

- RSA implementation explanation
- AES implementation explanation
- Hybrid encryption workflow
- Encoding scheme explanation
- Algorithms used
- Key generation process
- UI screenshots
- Business model explanation
- System architecture and workflow
- Member contributions, references, optimizations, and simplifications
- AI Usage Disclosure

Business Model Explanation

The selected business model is a Secure File Storage System. The user uploads a private file, and the application stores it as an encrypted bundle rather than storing the plaintext file.

Each uploaded file receives a unique AES-128 session key. AES encrypts the actual file contents, while RSA encrypts the AES session key using the user's public key. During decryption, the RSA private key recovers the AES key, and AES decrypts the file.

Real-World Scenario

- A user needs to store sensitive files safely.
- The system encrypts large file content efficiently with AES.
- The system protects the small AES session key with RSA.
- The UI demonstrates key generation, encryption, encrypted bundle download, decryption, and recovered file download.

System Architecture and Workflow

1. The user interacts with the Flask web UI.
2. The Flask route receives an uploaded file.
3. The hybrid layer generates a fresh AES session key and IV.
4. The AES module encrypts the file using AES-128-CBC.
5. The RSA module encrypts the AES session key using the RSA public key.
6. The application stores the encrypted AES key, IV, ciphertext, and metadata in a JSON .enc.txt bundle.
7. During decryption, the process is reversed using the RSA private key and recovered AES key.

Layer	Responsibility
UI Layer	Flask pages for keys, encryption, decryption, and downloads.
Hybrid Layer	Coordinates AES file encryption and RSA AES-key encryption.
AES Module	Implements AES-128 rounds, key expansion, CBC mode, and PKCS#7 padding.
RSA Module	Implements prime generation, key generation, modular exponentiation, and encryption/decryption.
Encoding Module	Converts binary encrypted values to Base64, Hex, and UTF-8 text where needed.

AES Implementation Explanation

AES-128 is implemented manually in the aes package. The implementation includes all required AES transformations: SubBytes, ShiftRows, MixColumns, AddRoundKey, and key expansion.

Component	File	Purpose
S-box and GF math	aes/sbox.py	Builds the AES S-box, inverse S-box, finite-field multiplication, and RCON values.
Key expansion	aes/key_expansion.py	Expands one 16-byte AES-128 key into 11 round keys.
AES block cipher	aes/aes_core.py	Encrypts and decrypts one 16-byte block using full AES rounds.
CBC mode	aes/modes.py	Encrypts arbitrary-length files using CBC mode and PKCS#7 padding.

AES Round Structure

- Initial AddRoundKey is applied before the main rounds.
- Rounds 1 through 9 apply SubBytes, ShiftRows, MixColumns, and AddRoundKey.
- Round 10 applies SubBytes, ShiftRows, and AddRoundKey without MixColumns.
- AES decryption applies inverse operations in reverse order.

RSA Implementation Explanation

RSA is implemented manually in the rsa package. The project includes prime generation, public/private key generation, encryption, decryption, modular inverse, and manual modular exponentiation.

Component	File	Purpose
GCD and EGCD	rsa/math_utils.py	Computes greatest common divisor and Bezout coefficients.
Modular inverse	rsa/math_utils.py	Computes d as the inverse of e modulo phi(n).
Modular exponentiation	rsa/math_utils.py	Uses repeated squaring instead of pow(a, b, m).
Prime generation	rsa/prime.py	Uses small-prime filtering and Miller-Rabin testing.
RSA core	rsa/rsa_core.py	Generates keys and encrypts/decrypts integers and AES-key bytes.

Key Generation Process

RSA Key Generation

8. Generate two distinct probable primes p and q .
9. Compute $n = p * q$.
10. Compute $\phi(n) = (p - 1) * (q - 1)$.
11. Use public exponent $e = 65537$.
12. Compute private exponent d as the modular inverse of e modulo $\phi(n)$.
13. Use (n, e) as the public key and (n, d) as the private key.

AES Session Key Generation

- For every uploaded file, the system generates a fresh 16-byte AES-128 key.
- The system also generates a fresh 16-byte IV for CBC mode.
- The AES key is not stored as plaintext inside the encrypted bundle.
- The AES key is encrypted with RSA and stored as Base64 text.

Hybrid Encryption Workflow

Phase	Steps
Encryption	Read file bytes; generate AES key and IV; encrypt file with AES-128-CBC; encrypt AES key with RSA public key; save JSON bundle.
Decryption	Read JSON bundle; Base64-decode fields; decrypt AES key with RSA private key; decrypt ciphertext with AES; save recovered file.

Encoding Scheme Explanation

Encrypted values are binary, so the project uses encoding schemes to display and transfer them safely in a text-based JSON bundle.

Encoding	Where Used	Reason
UTF-8	JSON metadata and text previews	Converts readable text to bytes and recovered bytes back to text.
Hex	AES key and IV display	Makes short binary values easy to inspect in the UI.
Base64	Encrypted AES key, IV, and ciphertext	Converts encrypted binary data into transferable text.
JSON	.enc.txt encrypted bundle	Stores metadata and Base64 encrypted fields in a readable structure.

Algorithms Used

Algorithm	Use in System
AES-128	Encrypts and decrypts file data.
CBC mode	Chains AES blocks using a random IV.
PKCS#7 padding	Pads plaintext to a multiple of the AES block size.
RSA	Encrypts and decrypts the AES session key.
Miller-Rabin	Tests generated RSA prime candidates.
Euclidean algorithm	Computes GCD during RSA key generation.
Extended Euclidean algorithm	Computes the RSA modular inverse.
Repeated squaring	Performs modular exponentiation without using $\text{pow}(a, b, m)$.
Base64, Hex, UTF-8	Encodes binary and text data for display and transfer.

Optimizations and Simplifications

Optimizations

- Miller-Rabin is used instead of trial division for large RSA prime candidates.
- Small-prime filtering rejects simple composites before Miller-Rabin testing.
- Repeated squaring makes RSA modular exponentiation efficient.
- The hybrid design encrypts large data with AES and only the small AES key with RSA.

Simplifications

- The RSA keypair is stored in memory for the educational demo.
- The UI displays secret values to make the workflow visible during evaluation.
- RSA byte wrapping uses a simple length-prefixed educational format.
- AES-CBC is used for the course requirement; production systems should add authentication.

UI Screenshots

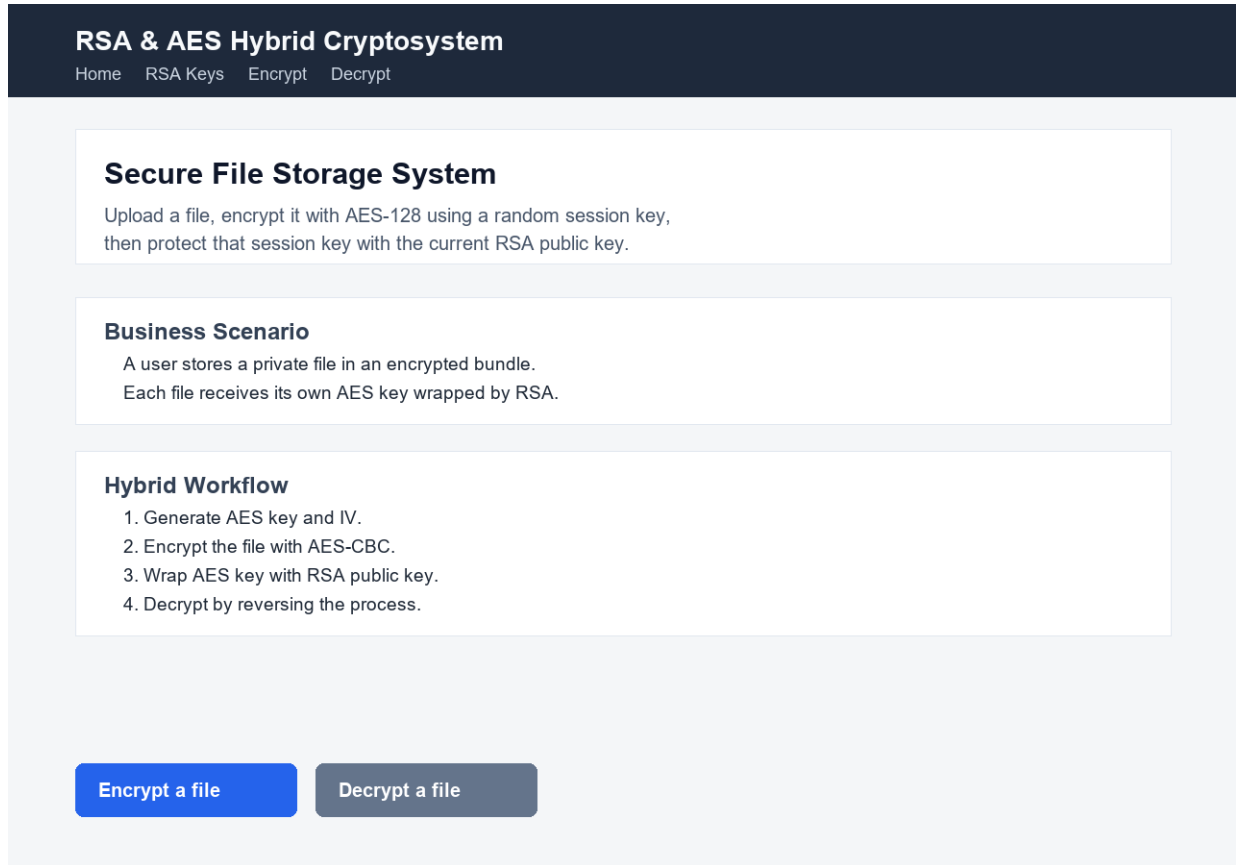


Figure 1: Home page showing the secure file storage scenario and hybrid workflow.

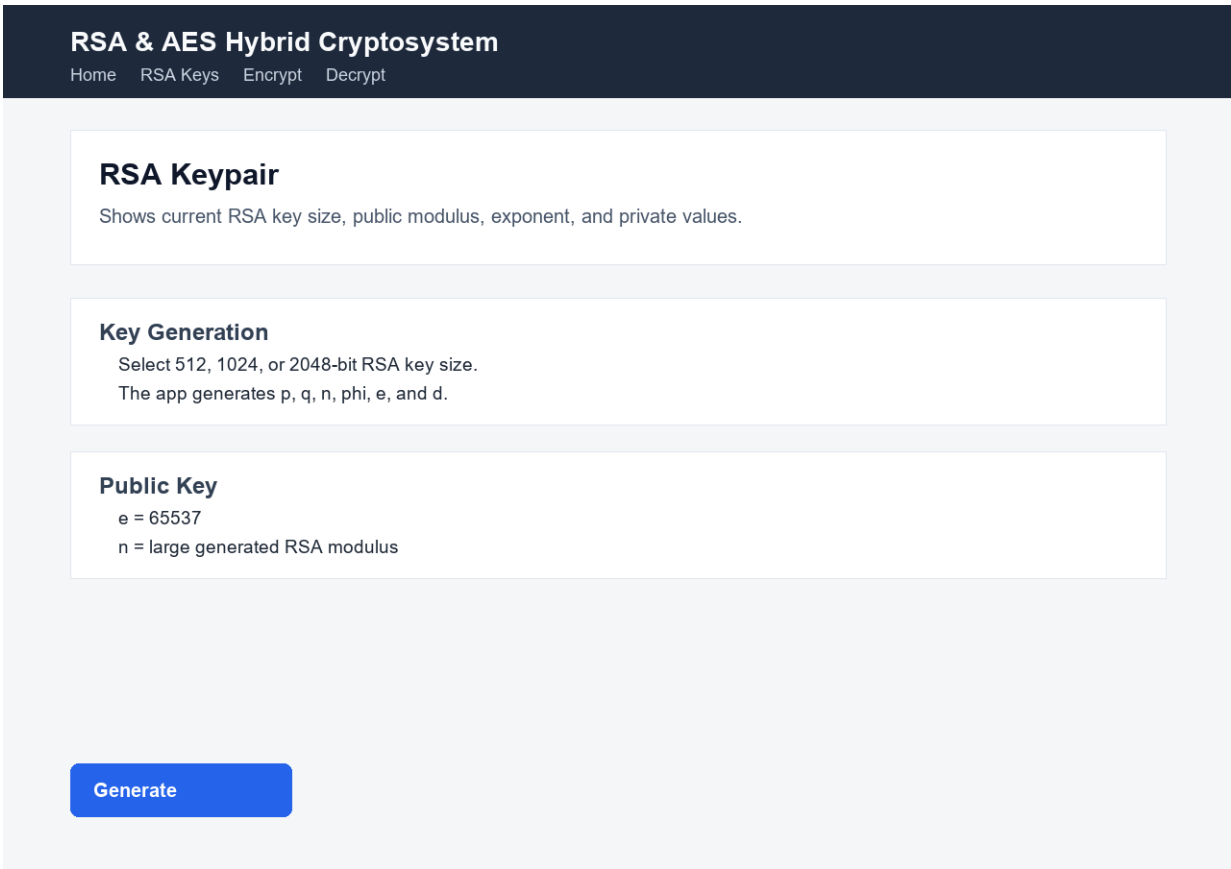


Figure 2: RSA key page showing key generation controls and key details.

Encrypt a File

Upload any file and create a downloadable encrypted bundle.

Encryption Actions

Generate random AES-128 session key and IV.

Encrypt file with AES-CBC.

Encrypt AES key with RSA public key.

Save fields as Base64 inside .enc.txt JSON.

Encrypt

Figure 3: Encryption page showing file upload and encrypted bundle creation.

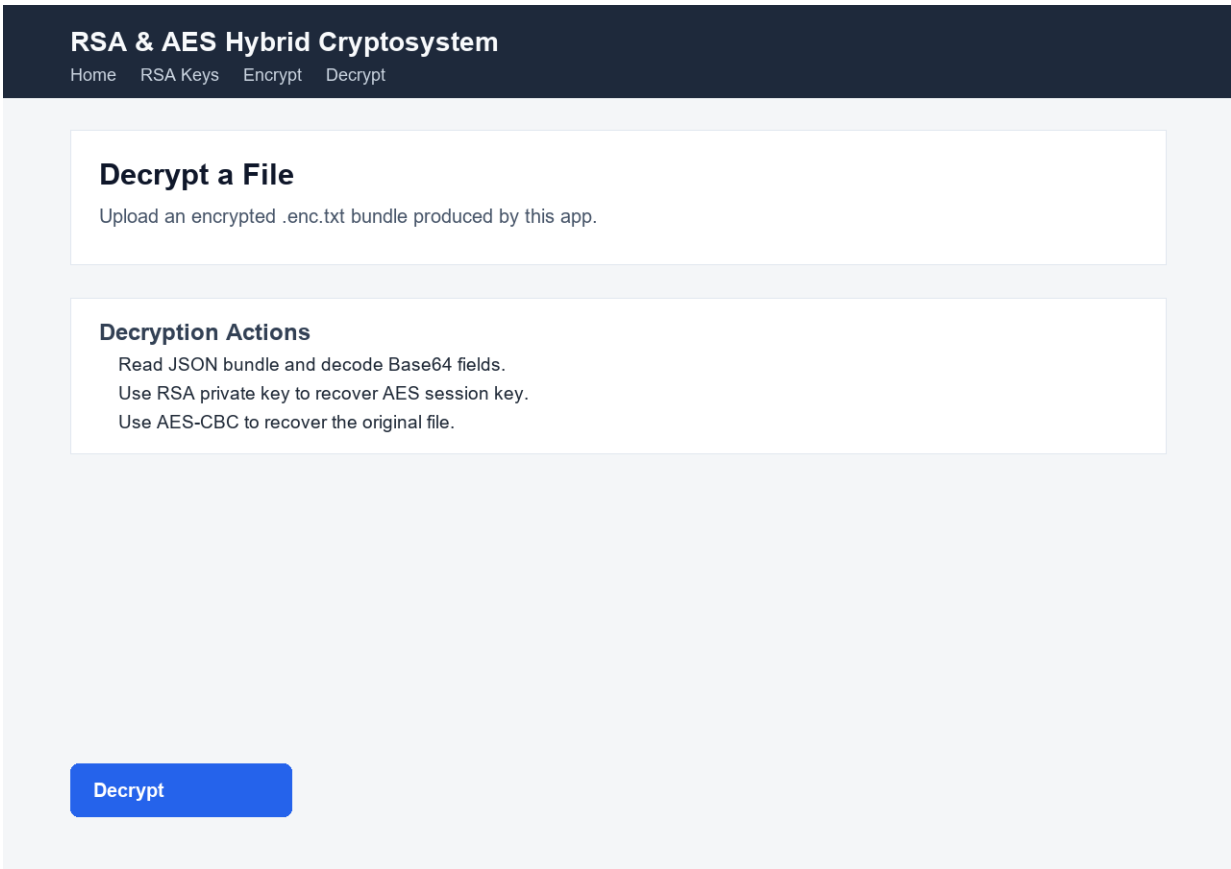


Figure 4: Decryption page showing encrypted bundle upload and recovery workflow.

Member Contributions

Actual member names were not provided, so the table uses placeholders that can be edited before submission.

Member	Implemented / Prepared Parts
Mohamed Ahmed 211006095	RSA math utilities, prime generation, RSA key generation, RSA encryption/decryption, and modular exponentiation.
Malak Ahmed 211004130	AES finite-field math, S-box generation, key expansion, AES block rounds, CBC mode, and padding.
Lina Ashraf 211004005	Hybrid cryptosystem layer, encryption/decryption workflow, encoding utilities, encrypted bundle format, and key-wrapping integration.
Sarah Adly 211005927	Flask UI pages and routes, file upload/download workflow, tests, README preparation, UI screenshots, and technical report.

References Used

- Course material and lecture notes for AES, RSA, and hybrid encryption.
- NIST FIPS 197: Advanced Encryption Standard (AES).
- RFC 8017: PKCS #1 RSA Cryptography Specifications.
- RFC 4648: Base64 Data Encodings.
- Flask documentation for routes, templates, file uploads, and downloads.
- Python documentation for secrets, json, base64, and unittest.

AI Usage Disclosure

Where AI Was Used

AI tools were used as an assistant during parts of the project. AI was used to help review the project requirements, organize the README and technical report, suggest documentation wording, and check whether the implemented files matched the required AES, RSA, hybrid cryptosystem, encoding, and UI components.

Why AI Was Used

AI was used to improve productivity and clarity. It helped summarize technical explanations, structure the report, identify the main implementation points that should be documented, and verify that the project clearly demonstrated the required cryptographic workflow.

AI-Generated Content or Code Modified by the Team

Any AI-generated documentation or suggested text was reviewed and modified by the team to match the actual project implementation. The README and technical report were edited so they describe only features that exist in the project. The business model explanation, workflow explanation, member contribution section, simplifications, and references were adjusted for the final submission.

Team Understanding of the Generated Code

The team understands the implemented code and workflow. AES-128-CBC encrypts uploaded file data using a unique AES session key and IV for each file. RSA encrypts the AES session key using the public key and recovers it using the private key during decryption. Base64, Hex, UTF-8, and JSON are used to represent encrypted binary data in readable and transferable formats.

The RSA implementation includes prime generation, public/private key generation, modular inverse, encryption/decryption, and manually written modular exponentiation using repeated squaring instead of Python `pow(a, b, m)`. The AES implementation includes key expansion, SubBytes, ShiftRows, MixColumns, AddRoundKey, AES rounds, CBC mode, and PKCS#7 padding.